



Taylor & Francis

Taylor & Francis Group

<http://taylorandfrancis.com>

7

Resources and the File System

Games are by nature multimedia experiences. A game engine therefore needs to be capable of loading and managing a wide variety of different kinds of media—texture bitmaps, 3D mesh data, animations, audio clips, collision and physics data, game world layouts, and the list goes on. Moreover, because memory is usually scarce, a game engine needs to ensure that only one copy of each media file is loaded into memory at any given time. For example, if five meshes share the same texture, then we would like to have only one copy of that texture in memory, not five. Most game engines employ some kind of *resource manager* (a.k.a. *asset manager*, a.k.a. *media manager*) to load and manage the myriad resources that make up a modern 3D game.

Every resource manager makes heavy use of the file system. On a personal computer, the file system is exposed to the programmer via a library of operating system calls. However, game engines often “wrap” the native file system API in an engine-specific API, for two primary reasons. First, the engine might be cross-platform, in which case the game engine’s file system API can shield the rest of the software from differences between different target hardware platforms. Second, the operating system’s file system API might not provide all the tools needed by a game engine. For example, many engines support file *streaming* (i.e., the ability to load data “on the fly” while the game is running), yet most operating systems don’t provide a streaming file system API

out of the box. Console game engines also need to provide access to a variety of removable and non-removable media, from memory sticks to optional hard drives to a DVD-ROM or Blu-ray fixed disk to network file systems (e.g., Xbox Live or the PlayStation Network, PSN). The differences between various kinds of media can likewise be “hidden” behind a game engine’s file system API.

In this chapter, we’ll first explore the kinds of file system APIs found in modern 3D game engines. Then we’ll see how a typical resource manager works.

7.1 File System

A game engine’s file system API typically addresses the following areas of functionality:

- manipulating file names and paths,
- opening, closing, reading and writing individual files,
- scanning the contents of a directory, and
- handling asynchronous file I/O requests (for streaming).

We’ll take a brief look at each of these in the following sections.

7.1.1 File Names and Paths

A *path* is a string describing the location of a file or directory within a file system hierarchy. Each operating system uses a slightly different path format, but paths have essentially the same structure on every operating system. A path generally takes the following form:

```
volume/directory1/directory2/.../directoryN/file-name
```

or

```
volume/directory1/directory2/.../directory(N - 1)/directoryN
```

In other words, a path generally consists of an optional *volume specifier* followed by a sequence of *path components* separated by a reserved path separator character such as the forward or backward slash (/ or \). Each component names a directory along the route from the root directory to the file or directory in question. If the path specifies the location of a file, the last component in the path is the file name; otherwise it names the target directory. The root directory is usually indicated by a path consisting of the optional volume specifier followed by a single path separator character (e.g., / on UNIX, or C : \ on Windows).

7.1.1.1 Differences across Operating Systems

Each operating system introduces slight variations on this general path structure. Here are some of the key differences between Microsoft DOS, Microsoft Windows, the UNIX family of operating systems and Apple Macintosh OS:

- UNIX uses a forward slash (/) as its path component separator, while DOS and older versions of Windows used a backslash (\) as the path separator. Recent versions of Windows allow either forward or backward slashes to be used to separate path components, although some applications still fail to accept forward slashes.
- Mac OS 8 and 9 use the colon (:) as the path separator character. Mac OS X is based on BSD UNIX, so it supports UNIX's forward slash notation.
- Some filesystems consider paths and filenames to be case-sensitive (like UNIX and its variants), while others are case-insensitive (like Windows). This can cause problems when dealing with files across multiple operating systems during development, or when writing a cross-platform game. (For example, should an asset file named `EnemyAnims.json` be considered equivalent to an asset file named `enemyanim.json` or not?)
- UNIX and its variants don't support volumes as separate directory hierarchies. The entire file system is contained within a single monolithic hierarchy, and local disk drives, network drives and other resources are *mounted* so that they appear to be subtrees within the main hierarchy. As a result, UNIX paths never have a volume specifier.
- On Microsoft Windows, volumes can be specified in two ways. A local disk drive is specified using a single letter followed by a colon (e.g., the ubiquitous `C:`). A remote network share can either be mounted so that it looks like a local disk, or it can be referenced via a volume specifier consisting of two backslashes followed by the remote computer name and the name of a shared directory or resource on that machine (e.g., `\\some-computer\some-share`). This double backslash notation is an example of the Universal Naming Convention (UNC).
- Under DOS and early versions of Windows, a file name could be up to eight characters in length, with a three-character *extension* which was separated from the main file name by a dot. The extension described the file's type, for example `.txt` for a text file or `.exe` for an executable file. In recent Windows implementations, file names can contain any number of dots (as they can under UNIX), but the characters after the

final dot are still interpreted as the file's extension by many applications including the Windows Explorer.

- Each operating system disallows certain characters in the names of files and directories. For example, a colon cannot appear anywhere in a Windows or DOS path except as part of a drive letter volume specifier. Some operating systems permit a subset of these reserved characters to appear in a path as long as the path is quoted in its entirety or the offending character is *escaped* by preceding it with a backslash or some other reserved *escape character*. For example, file and directory names may contain spaces under Windows, but such a path must be surrounded by double quotes in certain contexts.
- Both UNIX and Windows have the concept of a *current working directory* or CWD (also known as the *present working directory* or PWD). The CWD can be set from a command shell via the `cd` (*change directory*) command on both operating systems, and it can be queried by typing `cd` with no arguments under Windows or by executing the `pwd` command on UNIX. Under UNIX there is only one CWD. Under Windows, each volume has its own private CWD.
- Operating systems that support multiple volumes, like Windows, also have the concept of a *current working volume*. From a Windows command shell, the current volume can be set by entering its drive letter and a colon followed by the Enter key (e.g., `C : <Enter>`).
- Consoles often also employ a set of predefined path prefixes to represent multiple volumes. For example, PlayStation 3 uses the prefix `/dev_bdvd/` to refer to the Blu-ray disk drive, while `/dev_hddx/` refers to one or more hard disks (where *x* is the index of the device). On a PS3 development kit, `/app_home/` maps to a user-defined path on whatever host machine is being used for development. During development, the game usually reads its assets from `/app_home/` rather than from the Blu-ray or the hard disk.

7.1.1.2 Absolute and Relative Paths

All paths are specified relative to some location within the file system. When a path is specified relative to the root directory, we call it an *absolute path*. When it is relative to some *other* directory in the file system hierarchy, we call it a *relative path*.

Under both UNIX and Windows, absolute paths start with a path separator character (`/` or `\`), while relative paths have no leading path separator. On Windows, both absolute and relative paths may have an optional volume

specifier—if the volume is omitted, then the path is assumed to refer to the current working volume.

The following paths are all absolute:

Windows

- C:\Windows\System32
- D:\ (root directory on the D: volume)
- \ (root directory on the current working volume)
- \game\assets\animation\walk.anim (current working volume)
- \\joe-dell\Shared_Files\Images\foo.jpg (network path)

UNIX

- /usr/local/bin/grep
- /game/src/audio/effects.cpp
- / (root directory)

The following paths are all relative:

Windows

- System32 (relative to CWD \Windows on the current volume)
- X:animation\walk.anim (relative to CWD \game\assets on the X: volume)

UNIX

- bin/grep (relative to CWD /usr/local)
- src/audio/effects.cpp (relative to CWD /game)

7.1.1.3 Search Paths

The term *path* should not be confused with the term *search path*. A *path* is a string representing the location of a single file or directory within the file system hierarchy. A *search path* is a string containing a list of paths, each separated by a special character such as a colon or semicolon, which is searched when looking for a file. For example, when you run any program from a command prompt, the operating system finds the executable file by searching each directory on the search path contained in the shell's environment variable.

Some game engines also use search paths to locate resource files. For example, the OGRE rendering engine uses a resource search path contained in a text file named `resources.cfg`. The file provides a simple list of directories and ZIP archives that should be searched in order when trying to find an asset. That said, searching for assets at runtime is a time-consuming proposition. Usually there's no reason our assets' paths cannot be known a priori. Presuming this is the case, we can avoid having to search for assets at all—which is clearly a superior approach.

7.1.1.4 Path APIs

Clearly, paths are much more complex than simple strings. There are many things a programmer may need to do when dealing with paths, such as isolating the directory, filename and extension, canonicalizing a path, converting back and forth between absolute and relative paths and so on. It can be extremely helpful to have a feature-rich API to help with these tasks.

Microsoft Windows provides an API for this purpose. It is implemented by the dynamic link library `shlwapi.dll`, and it is exposed via the header file `shlwapi.h`. Complete documentation for this API is provided on the Microsoft Developer's Network (MSDN) at the following URL: [http://msdn2.microsoft.com/en-us/library/bb773559\(VS.85\).aspx](http://msdn2.microsoft.com/en-us/library/bb773559(VS.85).aspx).

Of course, the `shlwapi` API is only available on Win32 platforms. Sony provides similar APIs for use on the PlayStation 3 and PlayStation 4. But when writing a cross-platform game engine, we cannot use platform-specific APIs directly. A game engine may not need all of the functions provided by an API like `shlwapi` anyway. For these reasons, game engines often implement a stripped-down path-handling API that meets the engine's particular needs and works on every operating system targeted by the engine. Such an API can be implemented as a thin wrapper around the native API on each platform or it can be written from scratch.

7.1.2 Basic File I/O

The C standard library provides two APIs for opening, reading and writing the contents of files—one buffered and the other unbuffered. Every file I/O API requires data blocks known as *buffers* to serve as the source or destination of the bytes passing between the program and the file on disk. We say a file I/O API is *buffered* when the API manages the necessary input and output data buffers for you. With an unbuffered API, it is the responsibility of the programmer using the API to allocate and manage the data buffers. The C standard library's buffered file I/O routines are sometimes referred to as the *stream I/O* API, because they provide an abstraction which makes disk files look like streams of bytes.

The C standard library functions for buffered and unbuffered file I/O are listed in Table 7.1.

The C standard library I/O functions are well-documented, so we will not repeat detailed documentation for them here. For more information, please refer to <http://msdn.microsoft.com/en-us/library/c565h7xx.aspx> for Microsoft's implementation of the buffered (stream I/O) API, and to <http://msdn.microsoft.com/en-us/library/40bbyw78.aspx> for Microsoft's imple-

Operation	Buffered API	Unbuffered API
Open a file	<code>fopen()</code>	<code>open()</code>
Close a file	<code>fclose()</code>	<code>close()</code>
Read from a file	<code>fread()</code>	<code>read()</code>
Write to a file	<code>fwrite()</code>	<code>write()</code>
Seek to an offset	<code>fseek()</code>	<code>seek()</code>
Return current offset	<code>ftell()</code>	<code>tell()</code>
Read a single line	<code>fgets()</code>	n/a
Write a single line	<code>fputs()</code>	n/a
Read formatted string	<code>fscanf()</code>	n/a
Write formatted string	<code>fprintf()</code>	n/a
Query file status	<code>fstat()</code>	<code>stat()</code>

Table 7.1. Buffered and unbuffered file operations in the C standard library.

mentation of the unbuffered (low-level I/O) API.

On UNIX and its variants, the C standard library's unbuffered I/O routes are native operating system calls. However, on Microsoft Windows these routines are merely wrappers around an even lower-level API. The Win32 function `CreateFile()` creates or opens a file for writing or reading, `ReadFile()` and `WriteFile()` read and write data, respectively, and `CloseFile()` closes an open file handle. The advantage to using low-level system calls as opposed to C standard library functions is that they expose all of the details of the native file system. For example, you can query and control the security attributes of files when using the Windows native API—something you cannot do with the C standard library.

Some game teams find it useful to manage their own buffers. For example, the *Red Alert 3* team at Electronic Arts observed that writing data into log files was causing significant performance degradation. They changed the logging system so that it accumulated its output into a memory buffer, writing the buffer out to disk only when it was filled. Then they moved the buffer dump routine out into a separate thread to avoid stalling the main game loop.

7.1.2.1 To Wrap or Not to Wrap

A game engine can be written to use the C standard library's file I/O functions or the operating system's native API. However, many game engines *wrap* the file I/O API in a library of custom I/O functions. There are at least three advantages to wrapping the operating system's I/O API. First, the engine programmers can guarantee identical behavior across all target platforms, even when native libraries are inconsistent or buggy on a particular platform.

Second, the API can be simplified down to only those functions actually required by the engine, which keeps maintenance efforts to a minimum. Third, extended functionality can be provided. For example, the engine's custom wrapper API might be capable of dealing with files on a hard disk, a DVD-ROM or Blu-ray disk on a console, files on a network (e.g., remote files managed by Xbox Live or PSN), and also with files on memory sticks or other kinds of removable media.

7.1.2.2 Synchronous File I/O

Both of the standard C library's file I/O libraries are *synchronous*, meaning that the program making the I/O request must wait until the data has been completely transferred to or from the media device before continuing. The following code snippet demonstrates how the entire contents of a file might be read into an in-memory buffer using the synchronous I/O function `fread()`. Notice how the function `syncReadFile()` does not return until all the data has been read into the buffer provided.

```
bool syncReadFile(const char* filePath,
                  U8* buffer,
                  size_t bufferSize,
                  size_t& rBytesRead)
{
    FILE* handle = fopen(filePath, "rb");
    if (handle)
    {
        // BLOCK here until all data has been read.
        size_t bytesRead = fread(buffer, 1,
                                bufferSize, handle);

        int err = ferror(handle); // get error if any

        fclose(handle);

        if (0 == err)
        {
            rBytesRead = bytesRead;
            return true;
        }
    }
    rBytesRead = 0;
    return false;
}

void main(int argc, const char* argv[])
{

```

```

    U8 testBuffer[512];
    size_t bytesRead = 0;

    if (syncReadFile("C:\\testfile.bin",
                    testBuffer, sizeof(testBuffer),
                    bytesRead))
    {
        printf("success: read %u bytes\n", bytesRead);
        // contents of buffer can be used here...
    }
}

```

7.1.3 Asynchronous File I/O

Streaming refers to the act of loading data in the background while the main program continues to run. Many games provide the player with a seamless, load-screen-free playing experience by streaming data for upcoming levels from the DVD-ROM, Blu-ray disk or hard drive while the game is being played. Audio and texture data are probably the most commonly streamed types of data, but any type of data can be streamed, including geometry, level layouts and animation clips.

In order to support streaming, we must utilize an *asynchronous* file I/O library, i.e., one which permits the program to continue to run while its I/O requests are being satisfied. Some operating systems provide an asynchronous file I/O library out of the box. For example, the Windows Common Language Runtime (CLR, the virtual machine upon which languages like Visual BASIC, C#, managed C++ and J# are implemented) provides functions like `System.IO.BeginRead()` and `System.IO.BeginWrite()`. An asynchronous API known as `fios` is available for the PlayStation 3 and PlayStation 4. If an asynchronous file I/O library is not available for your target platform, it is possible to write one yourself. And even if you don't have to write it from scratch, it's probably a good idea to wrap the system API for portability.

The following code snippet demonstrates how the entire contents of a file might be read into an in-memory buffer using an asynchronous read operation. Notice that the `asyncReadFile()` function returns immediately—the data is not present in the buffer until our callback function `asyncReadComplete()` has been called by the I/O library.

```

AsyncRequestHandle g_hRequest; // async I/O request handle
U8 g_asyncBuffer[512];        // input buffer

static void asyncReadComplete(AsyncRequestHandle hRequest);

```

```

void main(int argc, const char* argv[])
{
    // NOTE: This call to asyncOpen() might itself be an
    // asynchronous call, but we'll ignore that detail
    // here and just assume it's a blocking function.

    AsyncFileHandle hFile = asyncOpen(
        "C:\\testfile.bin");

    if (hFile)
    {
        // This function requests an I/O read, then
        // returns immediately (non-blocking).
        g_hRequest = asyncReadFile(
            hFile,                // file handle
            g_asyncBuffer,        // input buffer
            sizeof(g_asyncBuffer), // size of buffer
            asyncReadComplete);    // callback function
    }

    // Now go on our merry way...
    // (This loop simulates doing real work while we wait
    // for the I/O read to complete.)

    for (;;)
    {
        OutputDebugString("zzz...\n");
        Sleep(50);
    }
}

// This function will be called when the data has been read.
static void asyncReadComplete(AsyncRequestHandle hRequest)
{
    if (hRequest == g_hRequest
        && asyncWasSuccessful(hRequest))
    {
        // The data is now present in g_asyncBuffer[] and
        // can be used. Query for the number of bytes
        // actually read:
        size_t bytes = asyncGetBytesReadOrWritten(
            hRequest);

        char msg[256];
        snprintf(msg, sizeof(msg),
            "async success, read %u bytes\n",

```

```

        bytes);
    OutputDebugString(msg);
}
}

```

Most asynchronous I/O libraries permit the main program to wait for an I/O operation to complete some time after the request was made. This can be useful in situations where only a limited amount of work can be done before the results of a pending I/O request are needed. This is illustrated in the following code snippet.

```

U8 g_asyncBuffer[512]; // input buffer

void main(int argc, const char* argv[])
{
    AsyncRequestHandle hRequest = ASYNC_INVALID_HANDLE;
    AsyncFileHandle hFile = asyncOpen(
        "C:\\testfile.bin");

    if (hFile)
    {
        // This function requests an I/O read, then
        // returns immediately (non-blocking).
        hRequest = asyncReadFile(
            hFile,                // file handle
            g_asyncBuffer,        // input buffer
            sizeof(g_asyncBuffer), // size of buffer
            nullptr);             // no callback
    }

    // Now do some limited amount of work...
    for (int i = 0; i < 10; i++)
    {
        OutputDebugString("zzz...\n");
        Sleep(50);
    }

    // We can't do anything further until we have that
    // data, so wait for it here.
    asyncWait(hRequest);

    if (asyncWasSuccessful(hRequest))
    {
        // The data is now present in g_asyncBuffer[] and
        // can be used. Query for the number of bytes
        // actually read:
        size_t bytes = asyncGetBytesReadOrWritten(
            hRequest);
    }
}

```



```

        char msg[256];
        snprintf(msg, sizeof(msg),
            "async success, read %u bytes\n",
            bytes);
        OutputDebugString(msg);
    }
}

```

Some asynchronous I/O libraries allow the programmer to ask for an estimate of how long a particular asynchronous operation will take to complete. Some APIs also allow you to set *deadlines* on a request (which effectively prioritizes the request relative to other pending requests), and to specify what happens when a request misses its deadline (e.g., cancel the request, notify the program and keep trying, etc.)

7.1.3.1 Priorities

It's important to remember that file I/O is a real-time system, subject to deadlines just like the rest of the game. Therefore, asynchronous I/O operations often have varying priorities. For example, if we are streaming audio from the hard disk or Blu-ray and playing it on the fly, loading the next buffer-full of audio data is clearly higher priority than, say, loading a texture or a chunk of a game level. Asynchronous I/O systems must be capable of suspending lower-priority requests, so that higher-priority I/O requests have a chance to complete within their deadlines.

7.1.3.2 How Asynchronous File I/O Works

Asynchronous file I/O works by handling I/O requests in a separate thread. The main thread calls functions that simply place requests on a queue and then return immediately. Meanwhile, the I/O thread picks up requests from the queue and handles them sequentially using blocking I/O routines like `read()` or `fread()`. When a request is completed, a callback provided by the main thread is called, thereby notifying it that the operation is done. If the main thread chooses to wait for an I/O request to complete, this is handled via a *semaphore*. (Each request has an associated semaphore, and the main thread can put itself to sleep waiting for that semaphore to be signaled by the I/O thread upon completion of the request. See Section 4.6.4 for more on semaphores.)

Virtually *any* synchronous operation you can imagine can be transformed into an asynchronous operation by moving the code into a separate thread—or by running it on a physically separate processor, such as on one of the CPU cores on the PlayStation 4. See Section 8.6 for more details.